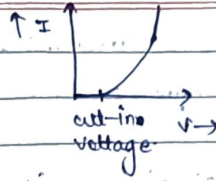


23/07/24

DIGITAL LOGIC...

→  (2 terminal diode).
acts as a switch.

 forward bias



$$V_{si} = 0.69V$$

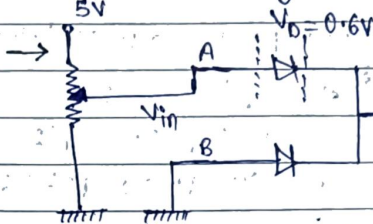
$$V_{ves} = 0.3V$$

→ from 0V to 0.6V $\Rightarrow$ 0's output i.e., Low level logic.

High level logic.

$$0.4V \rightarrow 5V$$

→  reverse bias
almost no current flows.

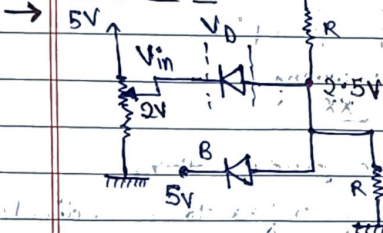


$$V_0 + V_0 = V_{in}$$

$$V_0 = V_{in} - V_D$$

$$\Rightarrow \text{for } V_{in} = 2V \Rightarrow V_0 = 2 - 0.6 = \underline{1.4V} \quad \text{OR Gate.}$$

AND GATE:

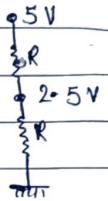


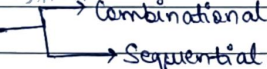
$$\text{If } V_{in} = 0V \Rightarrow V_0 = 0.6V$$

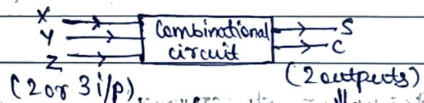
$$= 0.5V \Rightarrow V_0 = 1.1V$$

$$= 2V \Rightarrow V_0 = 2.5V$$

Now as $V_{in} \uparrow \Rightarrow V_0 = 2.5V$ because diode does not conduct.

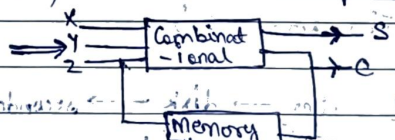
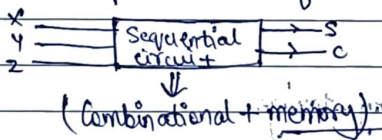


→ Circuit: 



(o/p depends on present i/p as well as present state of the circuit)

(depends only on present input)



25/07/24

NUMBER SYSTEM...

Decimal, Binary, Roman, Non-weighted no. system

(0-9) $\rightarrow$ {0, 1, 2, 3, 4, 5, 6, 7, 8, 9}

Weighted no. system $\rightarrow$ Each position has a weight.

Signed and Unsigned Binary no. representation:

5V $\rightarrow$ 1S only 2 stable states (on/off) $\rightarrow$ 2 binary no. system can be used. (0, 1)

Transistor

→ Bit $\rightarrow$ single binary digit (0, 1), Nibble $\rightarrow$ Collection of 4 bits.

Byte $\rightarrow$ Collection of 8 bits, Word $\rightarrow$ Collection of 16/32/64 bits.

→ For n bits, $b_{n-1} b_{n-2} \dots b_3 b_2 b_1 b_0$
 $2 \times 2 \times \dots \times 2 \times 2 \times 2 \times \dots \times 2 = 2^n$ possible cases.

→ For unsigned: If n bits $\Rightarrow$ minimum value = 0, maximum value = $2^n - 1$.

Signed Integer Representation:

- ① Signed-Magnitude Representation, ② 1's-complement representation,
③ 2's-complement Representation.

→ Signed Magnitude Representation: For n-bit no. representation, the most significant bit represents sign. (0 for +ve and 1 for -ve). Rest (n-1) bits represent magnitude of the number.

Max, Minimum no. = $-(2^{n-1}-1)$ to $+(2^{n-1}-1)$. $\Rightarrow$ Range

Ex. $1001 = \underline{\underline{-1}}$

→ 1's Complement: +ve no. is represented similar to signed magnitude and -ve no. are represented in 1's complement form.

Flip the bit and 1's complement is obtained. Most significant bit represents sign. ($0101 \rightarrow 1010$).

Range same as above.

Advantage: Subtraction can be done using addition. So, cost is low.

$(000) \rightarrow +0$ $(111) \rightarrow -0$ disadvantage (2 0s).

→ 2's Complement: +ve no. same as above, -ve in 2's complement form.

Add 1 to 1's complement $\Rightarrow$ 2's complement is obtained.

Q7 last से आगे 0 वैसा ही रहने दो, एक 1 को छोड़ दो, rest all are flipped.

Ex:
$$\begin{array}{r} 0101 \\ 1010 \end{array} \begin{array}{l} 1's \\ 2's \end{array}$$

$$\begin{array}{r} 0101 \\ + 1010 \\ \hline 1111 \end{array}$$

Range: 1011

MSB LSB

Advantage:- (•) Only one zero.

(*) Subtraction can be done by adding.

MSB LSB Range:- $-(2^{n-1})$ to $+(2^{n-1})$. $\rightarrow$ because 1 zeroless.

(represents sign)

• Weighted number representation with MSB having weight 2^{n-1}
 $3 = 0011$

Ex. $0101 = 4+1 = \underline{5}$, $1101 = 8+5 = \underline{13}$ $\rightarrow \underline{1101} \xrightarrow{2^8}$ complement

→ Shift left by k positions with zero padding multiplies the no. by 2^k .

Ex 9 = 00001001 $\xrightarrow{\text{1st shift}}$ 00010010 (padding with 0)
 $\xrightarrow{\text{2nd shift}}$ 00100100 $\rightarrow$ 36. (9×2^2)

Page : 24
Date : / /

Using 2's complement: ① Obtain 2's complement of B (say $\bar{B}$).
② $R = A + \bar{B}$. - If carry present, ignore it, result is +ve.
- else result -ve, in 2's complement form.

Ex: $A = 0111$, $B = 0100 \Rightarrow \bar{B} = 1100 \Rightarrow A+B = 0111 + 1100 = 0011$

$B-A = 0100 - 1001 \rightarrow \bar{A}$

$1101 \rightarrow (0011)$ (Taken 2's complement).

BCD and GRAY CODE: (Binary Coded Decimal).

BCD - Each digit of decimal no. is shown by a 4-bit of its binary representation.

Ex: $291 = \underbrace{0010}_2 \underbrace{1001}_9 \underbrace{0001}_1$, $13.24 = \underbrace{0001}_1 \underbrace{0011}_3 \underbrace{0010}_2 \underbrace{0100}_4$.

→ Addition using BCD: When 2 BCD nos. are added, there may be needed for correction steps, where 6 (0110) will be added to one of the nibble. The correction is required when nibble is one of the 6 unused combination or there is a carry from the previous nibble. [1010, 1011, 1100, 1101, 1110, 1111] → [10 to 15]

Ex: (i) $23 + 46 = 69$. $23 = 0010\ 0011$, $46 = 0100\ 0110$

⇒ $\begin{array}{r} 0010\ 0011 \\ 0100\ 0110 \\ \hline 0110\ 1001 \end{array}$ (1 nibble = 4-bit no.) (No carry from 1 nibble to other).

(adding 6). $\begin{array}{r} 0110\ 1011 \\ \leftarrow 0110 \\ \hline 71 = 0111\ 1001 \end{array}$ (This nibble unused as 11)

(iii) $28 + 39 = 67$. $\begin{array}{r} 0010\ 1000 \\ 0011\ 1001 \\ \hline 0110\ 0001 \end{array}$ (Carry from 1 nibble to other).

+ 0110 (adding 6).
 $\hline 0110\ 0111 \equiv 67$.

→ In some applications, consecutive no. differ by multiple bits due to which error may rise, but in Gray Code change of only 1 bit b/w successive numbers. It reduces error.

Gray Code - Self-reflecting code; successive code word changes by 1 bit.

(2-bit Gray code) $\begin{array}{l} 00 \\ 01 \\ 11 \\ 10 \end{array}$ $\xrightarrow[\text{Image}]{\text{Mirror}}$ $\begin{array}{l} 000 \\ 001 \\ 011 \\ 010 \end{array}$ (To convert to 3-bit) Take image, add 0 to 1st and 1 to next part.

(mirror) $\begin{array}{l} 110 \\ 111 \\ 101 \\ 100 \end{array}$ (mirror image of above)

Commutative and Associative law:

$$X \cdot Y = Y \cdot X, \quad X + Y = Y + X \quad \text{Commutative}$$

$$(X \cdot Y) \cdot Z = X \cdot (Y \cdot Z), \quad (X + Y) + Z = X + (Y + Z) \quad \text{Associative}$$

$$\text{Distributive Laws: } X \cdot (Y + Z) = (X \cdot Y) + (X \cdot Z)$$

$$X + Y \cdot Z = (X + Y) \cdot (X + Z)$$

$$\rightarrow \text{Simplification Laws: } X \cdot X + X \cdot \bar{X} = X, \quad (X + Y)(X + \bar{Y}) = X, \\ X + X \cdot Y = X, \quad X(X + Y) = X, \quad (X + \bar{X}) \cdot Y = X \cdot Y, \quad X + \bar{X} \cdot Y = X + Y$$

$$\text{Proof: } X + X \cdot Y = X \cdot 1 + X \cdot Y = X(1 + Y) = X \cdot 1 = X$$

$$\rightarrow \text{DeMorgan's Laws: } \overline{(X + Y)} = \bar{X} \cdot \bar{Y}, \quad \overline{(X \cdot Y)} = \bar{X} + \bar{Y}$$

$\rightarrow$ Duality - Given a boolean function, the dual is formed by replacing AND with OR, OR with AND, 0 with 1, 1 with 0. The variable and complement are unchanged.

$$\text{Ex: } \bar{a} \cdot \bar{b} + c + 0 \cdot d(1 + c) = F \Rightarrow F^D = (\bar{a} + \bar{b}) \cdot c \cdot (1 + d + 0 \cdot c)$$

$\rightarrow$ The Consensus Theorem: This theorem is useful for simplifying boolean expression.

$$XY + \bar{X}Z + YZ = XY + \bar{X}Z \quad (YZ = \text{redundant})$$
(the redundant term YZ is called 'consensus term')

- Given a pair of terms, a variable appears in 1 term and its complement appear in other term, then product of these terms leaving out the variable and its complement, then the remainder is known as consensus term.

$$\rightarrow \text{Dual form of consensus theorem: } (X + Y)(\bar{X} + Z)(Y + Z) = (X + Y)(\bar{X} + Z)$$

$$\text{Proof: } XY + \bar{X}Z + YZ = XY + \bar{X}Z + (X + \bar{X})YZ = XY + X \cdot YZ + \bar{X}Z + \bar{X} \cdot YZ \\ = XY(1 + Z) + \bar{X}Z(1 + Y) = XY \cdot 1 + \bar{X}Z \cdot 1 = XY + \bar{X}Z$$

$$\text{Ex: } F = \bar{a}\bar{b} + ac + bc + \bar{b}c + ab = \bar{a}\bar{b} + ac + \underline{bc} \quad (\text{Take 2 terms at once, check and eliminate})$$

$\uparrow \quad \uparrow \quad \uparrow$
 Remove $\bar{b}c$ Remove ab

$\rightarrow$ Simplification of boolean expressions using boolean algebra:

3 basic ways - (1) Combining terms, (2) Eliminating terms,

(3) Eliminating literals.

(Use theorem $X + X \cdot Y = X + Y$ to combine 2 terms)

(Use theorem $X + \bar{X} \cdot Y = X + Y$ to eliminate the redundant literals, further terms can be added or multiplied)

(Use theorem $X + X \cdot Y = X + Y$ to eliminate redundant term)

if possible apply the consensus theorem).

$$\text{Ex: } \bar{a} \bar{b} \bar{c} d + a \bar{b} \bar{c} d = \underline{\bar{a} \bar{b} d}$$

Eliminate

$$\begin{aligned}
 &= \bar{A}\bar{C}\bar{D} + \bar{A}BD + ABCD + A\bar{C}\bar{D} + \bar{B}\bar{C}\bar{D} = \bar{A}\bar{C}\bar{D} + BD(\bar{A}+A) + A\bar{C}\bar{D} + \bar{B}\bar{C}\bar{D} \\
 &= \bar{A}\bar{C}\bar{D} + BD(\bar{A}+1) + A\bar{C}\bar{D} + \bar{B}\bar{C}\bar{D} = \bar{A}\bar{C}\bar{D} + \bar{A}BD + BCD + A\bar{C}\bar{D} + \bar{B}\bar{C}\bar{D} \\
 &= \bar{A}\bar{C}\bar{D} + \bar{A}BD + \cancel{BCD} + \cancel{A\bar{C}\bar{D}} + \bar{B}\bar{C}\bar{D} + \cancel{ABC} \quad (\text{adding a redundant term}) \\
 &= \bar{A}\bar{C}\bar{D} + \bar{A}BD + \bar{B}\bar{C}\bar{D} + ABC \quad (2 \text{ times consensus theorem}).
 \end{aligned}$$

- Ex. $\frac{\text{Marry watches TV}}{F} \text{ if } \frac{\text{it is Sunday}}{A} \text{ and } \frac{\text{she has finished her h.w.}}{B}$

$$\Rightarrow F = A.B$$

④ A logic circuit that has 3 i/p and 1 o/p. I/p are denoted by A, B, C. o/p is shown by F and $F = 1$ if $N \geq 011$. (N is no. formed by A, B, C), 0 otherwise.

A B C F F $\Rightarrow$ Truth table.

$$F = \bar{A}BC + A\bar{B}\bar{C} + A\bar{B}C + A\bar{B}\bar{C} + ABC$$

$$F = BC + AB + A\bar{B}\bar{C}$$

$$101 \quad 10 \quad \Rightarrow F = BC + A(\bar{B} + \bar{C}) = BC + A(\bar{B} + \bar{C})$$

$$\Rightarrow F = BC + A\bar{B} + A\bar{C} + \underbrace{AB}_{\substack{\text{Addition} \\ = A+B.C.}} = BC + A\bar{C} + A = BC + A(\bar{C} + 1)$$

→ Combinational Circuit:

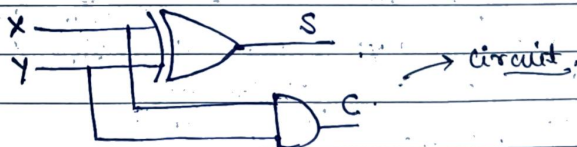
The o/p of a combinational circuit depend on present combination of i/p only.

- Half Adder: 2 i/p and 2 o/p. (2-bit numbers is added).

X	Y	S	C
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

$$S = x\bar{y} + \bar{x}y = x \oplus y \quad (\text{XOR gate})$$

$$C = x \cdot y$$



- Full Adder : Combinational ckt. that performs addition of 3 i/p bits, 2 outputs.

$$\begin{aligned} (x \oplus y) &= x\bar{y} + \bar{x}y = (\bar{x}+y) \cdot (x+\bar{y}) = \bar{x}\bar{x} + \bar{x}y + xy + y\bar{y} \\ &= \bar{x}y + xy \end{aligned}$$

X	Y	Z	S	C
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

$$S = \bar{x}\bar{y}z + \bar{x}y\bar{z} + x\bar{y}\bar{z} + x y z$$

$$= z(x\bar{y} + \bar{x}y) + \bar{z}(x\bar{y} + x y)$$

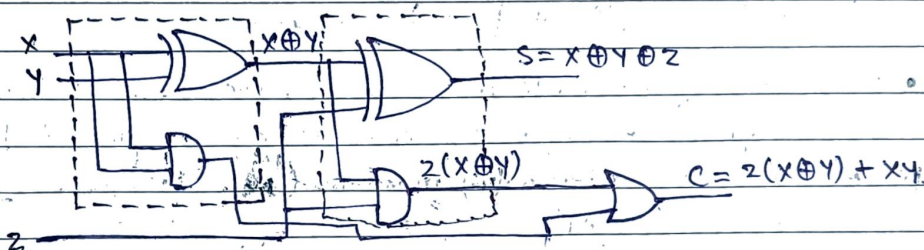
$$= z(x \oplus y) + \bar{z}(x \oplus y)$$

$$= x \oplus y \oplus z$$

$$C = x y z + x y \bar{z} + x \bar{y} z + x \bar{y} \bar{z}$$

$$C = z(x \oplus y) + x y$$

Circuit Diagram:

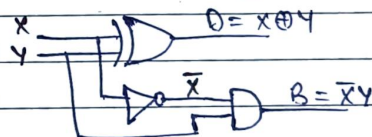


- Half Subtractor: 2 bit subtraction at a time, 2 output (diff. and borrow)

X	Y	D	B
0	0	0	0
1	1	0	0
1	0	1	0
0	1	1	1

$$D = \bar{x}y + x\bar{y} = x \oplus y$$

$$B = \bar{x}y$$



Circuit

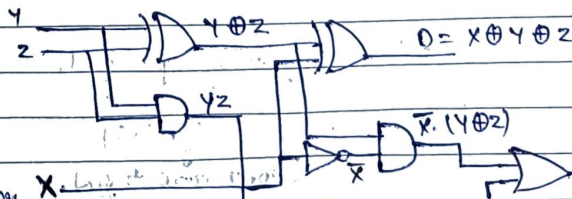
- Full Subtractor: Performs subtraction of 2 bits, taking into consideration, a 1 have been borrowed by the lower significant stage.

X	Y	Z	D	B
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

$$D = \bar{x}\bar{y}z + \bar{x}y\bar{z} + x\bar{y}\bar{z} + x y z = x \oplus y \oplus z$$

$$B = \bar{x}\bar{y}z + \bar{x}y\bar{z} + \bar{x}yz + x y \bar{z}$$

$$= \bar{x}\bar{y}z + \bar{x}y\bar{z} + \bar{x}yz = \bar{x}(y \oplus z) + yz$$



→ Minimum no. of NAND gates

for half adder = 5 and for

full adder = 9.

we have, $A+B+C=0$ if $A=B=C=0$; $A+B+C=0$ if $A=B=0, C=1$;
 $A+B+C=0$ if $A=0, B=1$ and $C=0$.

$$(x+y)(x+z) = x+yz.$$

$$f = \underbrace{(A+B+C)}_x \cdot \underbrace{(A+B+\bar{C})}_y \cdot \underbrace{(A+B+C)}_z = \underbrace{(A+B)}_x \cdot \underbrace{(A+B+C)}_y = A+B(\bar{B}+C) = A+BC$$

or, $\because x+yz = (x+y)(x+z)$.

$$f = \bar{A}\bar{B}\bar{C} + \bar{A}\bar{B}C + A\bar{B}\bar{C} \Rightarrow \text{Complement both sides:}$$

$$(\bar{f}) = \overline{(\bar{A}\bar{B}\bar{C} + \bar{A}\bar{B}C + A\bar{B}\bar{C})} \Rightarrow f = \overline{(\bar{A}\bar{B}\bar{C})} \cdot \overline{(\bar{A}\bar{B}C)} \cdot \overline{(A\bar{B}\bar{C})}$$

$$\Rightarrow f = (\bar{A} + \bar{B} + \bar{C}) \cdot (\bar{A} + B + \bar{C}) \cdot (A + \bar{B} + C) = (A+B+C) \cdot (A+\bar{B}+C) \cdot (A+B+\bar{C})$$

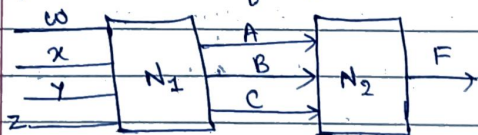
→ Minterms and Maxterms: In general, a minterm of n variables is a product of n literals, in which each variable appears exactly once in either true or complement form but not both.

• A maxterm of n variable is a sum of n literals, in which each variable appears exactly once in either true or complement form.

Ex

A	B	C	Minterms(m_i)	Maxterms(M_i)	$i = 0, 1, 2, \dots, 7$
0	0	0	$\bar{A}\bar{B}\bar{C}$	$A+B+C$	
0	0	1	$\bar{A}\bar{B}C$	$A+B+\bar{C}$	$f = m_3 + m_4 + m_5 + m_6 + m_7$
0	1	0	$\bar{A}B\bar{C}$	$A+\bar{B}+C$	$f(A,B,C) = \sum m(3,4,5,6,7)$
0	1	1	$\bar{A}BC$	$A+\bar{B}+\bar{C}$	or
1	0	0	$A\bar{B}\bar{C}$	$\bar{A}+B+C$	$f(A,B,C) = \prod M(0,1,2)$
1	0	1	$A\bar{B}C$	$\bar{A}+B+\bar{C}$	Standard sum-of-product form
1	1	0	$AB\bar{C}$	$\bar{A}+\bar{B}+C$	Standard Product sum-of-product form
1	1	1	ABC	$\bar{A}+\bar{B}+\bar{C}$	- of - Sum form

→ Incompletely Specified Function: Assume o/p of N_1 do not generate all possible values of i/p. Let: 001, 110 are not generated.



Here, F is incompletely specified.

$$F = \sum m(0, 3, 7) + \sum d(1, 6)$$

(don't care term)

A	B	C	F
0	0	0	1
0	0	1	<u>X</u>
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	<u>X</u>
1	1	1	1

$\bar{A}\bar{B}\bar{C}, A\bar{B}\bar{C}$ are don't care terms.

⑧ Find the minterm expansion: $f(a,b,c,d) = \bar{a}(\bar{b}+d) + ac\bar{d}$

$$f(a,b,c,d) = \bar{a}\bar{b} + \bar{a}d + ac\bar{d} = \bar{a}\bar{b}(c+\bar{c}) + \bar{a}d(b+\bar{b}) + ac\bar{d}(b+\bar{b})$$

$$\Rightarrow f = \bar{a}\bar{b}c + \bar{a}\bar{b}\bar{c} + \bar{a}bd + \bar{a}\bar{b}d + abc\bar{d} + a\bar{b}c\bar{d}$$

$$= \bar{a}\bar{b}c(d+\bar{d}) + \bar{a}\bar{b}\bar{c}(d+\bar{d}) + \bar{a}bd(c+\bar{c}) + \bar{a}\bar{b}d(c+\bar{c}) + abc\bar{d} + a\bar{b}c\bar{d}$$

$(\bar{a}+b+c+d)(\bar{a}+b\bar{c}+d)(\bar{a}+b\bar{c}+d)(\bar{a}+b\bar{c}+d)(\bar{a}+b\bar{c}+d)(\bar{a}+b\bar{c}+d)(\bar{a}+b\bar{c}+d)(\bar{a}+b\bar{c}+d)(\bar{a}+b\bar{c}+d)(\bar{a}+b\bar{c}+d)$
 $= (\bar{a}+b+c+d)(\bar{a}+b\bar{c}+d)(\bar{a}+b\bar{c}+d)(\bar{a}+b\bar{c}+d)(\bar{a}+b\bar{c}+d)(\bar{a}+b\bar{c}+d)(\bar{a}+b\bar{c}+d)(\bar{a}+b\bar{c}+d)(\bar{a}+b\bar{c}+d)(\bar{a}+b\bar{c}+d)$
 $(\bar{a}+b+c+d)(\bar{a}+b\bar{c}+d)(\bar{a}+b\bar{c}+d)(\bar{a}+b\bar{c}+d)(\bar{a}+b\bar{c}+d)(\bar{a}+b\bar{c}+d)(\bar{a}+b\bar{c}+d)(\bar{a}+b\bar{c}+d)(\bar{a}+b\bar{c}+d)(\bar{a}+b\bar{c}+d)$
[Remove 2 duplicates]

$= \bar{a}\bar{b}cd + \bar{a}b\bar{c}d + \bar{a}b\bar{c}d + \bar{a}b\bar{c}d + \bar{a}b\bar{c}d + \bar{a}b\bar{c}d + \bar{a}b\bar{c}d + \bar{a}b\bar{c}d + \bar{a}b\bar{c}d + \bar{a}b\bar{c}d$
 $+ \bar{a}b\bar{c}d = \sum m(0, 1, 2, 3, 5, 7, 10, 14)$ $(x+x=x \Rightarrow \text{duplicates removed})$

$\underline{\underline{AM(4, 6, 8, 9, 11, 12, 13, 15)}}$

⑧ Expand as product of max terms: $f(a, b, c, d) = (\bar{a}+c+d)(a+b+d)$
 Use $x+y+z = (x+y)(x+z)$, and then do $x \cdot \bar{x}$ for all missing.
 $\Rightarrow (\bar{a}+c)(\bar{a}+d)(a+b+d) \Rightarrow (\bar{a}+b\bar{b}+c+d)(\bar{a}+b\bar{b}+c\bar{c}+d)(a+b+c\bar{c}+d)$ ⑨
 $= \underline{\underline{AM(4, 6, 8, 9, 11, 12, 13, 15)}}$

(K-map).

→ KARNAUGH Map: Pictorial representation of truth table.

If n variable in function $\Rightarrow 2^n$ cells in map and each cell will represent one of the minterms. 2 adjacent cells differ in 1 variable, 2^2 adjacent cells differ in 2 variables $\Rightarrow 2^m$ adjacent cells differ in m variable.

Drawback: Difficult to visualize for > 4 variables.

Basic Approach: ① Fill up the cells of K-map with true minterms of the function. ② Group the true minterms of the function into cubes such that - (i) the size of cubes are maximized, (ii) every minterm covered by at least one cube.

③ Write down the minimized sum of product expression for the function by creating one product out of every cube that has been selected.

Ex. (i) $f = \bar{x}y + x\bar{y} + xy \Rightarrow f = x+y$
 (ii) $f = \bar{x}yz + \bar{x}y\bar{z} + \bar{x}\bar{y}z + \bar{x}\bar{y}\bar{z} \Rightarrow f = \bar{x}$

(iii) $f = \bar{x}yz + \bar{x}y\bar{z} + \bar{x}\bar{y}z + \bar{x}\bar{y}\bar{z} \Rightarrow f = \bar{x}y + \bar{x}\bar{y} = \bar{x}$
 (iv) $f = \bar{x}yz + \bar{x}y\bar{z} + \bar{x}\bar{y}z + \bar{x}\bar{y}\bar{z} \Rightarrow f = \bar{x}y + \bar{x}\bar{y} = \bar{x}$
 (v) $f = \bar{x}yz + \bar{x}y\bar{z} + \bar{x}\bar{y}z + \bar{x}\bar{y}\bar{z} \Rightarrow f = \bar{x}y + \bar{x}\bar{y} = \bar{x}$

→ for 4 variables: 2 adjacent cells represent term of 3 variables, 4 cells → 2 variables, 8 cells → 1 variable.

wx \ yz

	00	01	11	10
00	m_0	m_1	m_3	m_2
01	m_4	m_5	m_7	m_6
11	m_{12}	m_{13}	m_{15}	m_{14}
10	m_8	m_9	m_{11}	m_{10}

Ex: $F(w, x, y, z) = \sum m(0, 1, 2, 4, 5, 6, 8, 9, 12, 13, 14)$

wx \ yz

00	01	11	10
00	1	1	1
01	1	1	1
11	1	1	1
10	1	1	1

Annotations: wz (pointing to top row), xz (pointing to right column), y (pointing to middle row).

$= \bar{y} + w\bar{z} + xz$

① $F = \bar{A}\bar{B}\bar{C} + \bar{B}\bar{C}\bar{D} + \bar{A}\bar{B}C\bar{D} + \bar{A}\bar{B}\bar{C}$

$= \bar{A}\bar{B}\bar{C}(D + \bar{D}) + \bar{A}\bar{B}C\bar{D} + \bar{A}\bar{B}\bar{C}\bar{D} + \bar{A}\bar{B}\bar{C}D + \bar{A}\bar{B}\bar{C}\bar{D}$

$= \bar{A}\bar{B}\bar{C}\bar{D} + \bar{A}\bar{B}\bar{C}\bar{D} + \bar{A}\bar{B}C\bar{D} + \bar{A}\bar{B}\bar{C}\bar{D} + \bar{A}\bar{B}\bar{C}D + \bar{A}\bar{B}\bar{C}\bar{D}$

wx \ yz

00	01	11	10
00	1	1	1
01	1	1	1
11	1	1	1
10	1	1	1

Annotations: $\bar{A}\bar{B}\bar{C}$ (pointing to top row), $\bar{B}\bar{C}$ (pointing to middle row), $\bar{A}\bar{B}$ (pointing to left column), $\bar{A}\bar{B}\bar{C}$ (pointing to top-left cell), $\bar{A}\bar{B}\bar{C}$ (pointing to top-right cell), $\bar{A}\bar{B}\bar{C}$ (pointing to bottom-left cell), $\bar{A}\bar{B}\bar{C}$ (pointing to bottom-right cell).

$\Rightarrow \bar{A}\bar{B}\bar{C} + \bar{A}\bar{B}\bar{C} + \bar{A}\bar{B}\bar{C} + \bar{A}\bar{B}\bar{C}$

$= \bar{B}\bar{C} + \bar{A}\bar{B} + \bar{A}\bar{C}$

② $F(w, x, y, z) = \sum m(1, 3, 7, 11, 15) + \sum d(0, 2, 5)$

wx \ yz

00	01	11	10
00	x	1	1
01	x	1	1
11	1	1	1
10	1	1	1

Not compulsory to include all don't care terms. If by including it cube size increases, then take it. Try to take minimum no. of don't care terms.
 $\Rightarrow F = yz + w\bar{z}$

→ Determination of Minimum Expression Using Essential Prime Implicant:

Implicant - Any single 1 or any group of 1's which can be combined together on a map of a function 'f' represents a product term is called implicant of 'f'.

Ex: $abc \rightarrow abc$ (Prime Implicant)

wx \ yz

00	01	11	10
00	1	1	1
01	1	1	1
11	1	1	1
10	1	1	1

Annotations: $\bar{a}\bar{b}\bar{c}$ (pointing to top-left cell), $\bar{a}\bar{b}c$ (pointing to top-right cell), $\bar{a}b\bar{c}$ (pointing to middle-left cell), $\bar{a}bc$ (pointing to middle-right cell), $\bar{a}\bar{c}$ (pointing to top row), $\bar{b}\bar{c}$ (pointing to middle row), $\bar{a}\bar{b}$ (pointing to left column).

Prime Implicant - An implicant is called prime implicant if it cannot be combined with other terms to eliminate variables.

Essential Prime Implicant - A prime implicant of a function is said to be essential prime implicant if it covers at least a minterm of the function that is not covered by any other prime implicant of the function.

Ex:

cd \ ab	00	01	11	10
00		1	1	
01	1	1	1	
11	1		1	1
10		1	1	1

Prime Implicant: $ab + b\bar{c} + ac + \bar{a}\bar{c}d + \bar{a}\bar{b}d$

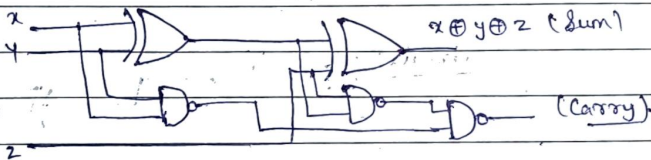
$\bar{b}cd$. (Total 6)

Essential Prime Implicant: $b\bar{c} + ac$. (Only 2)

→ All P.I. must be there in final expression. Other P.I. may be there if some minterm is left. $\Rightarrow F(a,b,c,d) = ac + b\bar{c} + \bar{a}\bar{b}d$.

• Full Adder:- By using only XOR, NAND:

Replace 2 AND by 4 NAND, and OR by 3 NAND. $\Rightarrow$ 7 NAND gates.
But if in a path continuously 2 NAND come, remove them as it becomes double inverter $\Rightarrow$ Only 3 NAND gates. [2 are inverters $\Rightarrow$ no change].



→ Exclusive OR and Exclusive NOR functions: $XOR: x \oplus y \Rightarrow \bar{x}y + x\bar{y}$

$XNOR: - x \odot y = \bar{x}y + xy$. (Both are commutative and associative).
complement of XOR: $A \oplus (B \oplus C) = (A \oplus B) \oplus C = A \oplus B \oplus C$.

These 2 are economical.

• An n-variable XOR expression is equal to the boolean function with $2^{n/2}$ minterm whose equivalent binary numbers has an odd no. of 1's.

Ex: $A \oplus B \oplus C \oplus D \Rightarrow \frac{2^4}{2} = 8$ minterms $\Rightarrow$ each with odd no. of 1.

$$\Sigma m(1, 2, 4, 7, 11, 13, 14) = (\bar{A}\bar{B} + \bar{A}B) \oplus (C\bar{D} + \bar{C}D)$$

$$= (\bar{A}\bar{B} + \bar{A}B) \cdot (C\bar{D} + \bar{C}D) + (\bar{A}\bar{B} + \bar{A}B) \cdot (C\bar{D} + \bar{C}D) = (\bar{A}\bar{B} + \bar{A}B) \cdot (\bar{C}\bar{D} + CD) +$$

$$= \bar{A}\bar{B}\bar{C}\bar{D} + \bar{A}\bar{B}C\bar{D} + \bar{A}B\bar{C}\bar{D} + \bar{A}B\bar{C}D + \bar{A}B\bar{C}\bar{D} + \bar{A}B\bar{C}D + A\bar{B}\bar{C}\bar{D} + A\bar{B}\bar{C}D$$

• An n-variable XNOR expression is equal to the boolean function with $2^{n/2}$ minterms whose equivalent binary no.s have even no. of 0's.

Ex: $A \odot B \odot C \odot D \Rightarrow 8$ terms $= \Sigma m(0, 3, 5, 6, 9, 10, 12, 15)$.

• When the no. of variables in a function is odd, the minterms with an even no. of 0's are same as the minterms with an odd no. of 1's.

Ex: Full adder (Sum Expression). $(A \oplus B \oplus C = A \odot B \odot C)$.

0 1 0 1
0 1 0 1
1 1 0 1

- When the minterms of a function with an odd no. of variable have an even nos of 1's or (odd no. of 0's), the function can be expressed as complement of XOR or complement of XNOR.

Ex. $(A \oplus B \oplus C)' = (A \oplus B) \odot C$; $(A \odot B \odot C)' = (A' \odot B) \oplus C$.

→ Odd Parity: (odd parity generators).

XOR chip = 74186 (4 XOR)

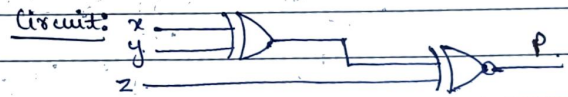
Three-bit message Parity bit.

x	y	z	P
0	0	0	1
0	1	0	0
0	0	1	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	0

add 1 if even no. of 1 are there.

$$P = \bar{x}y\bar{z} + \bar{x}yz + x\bar{y}z + xy\bar{z}$$

$$= (x \oplus y \oplus z)' = x \odot y \odot z$$



$\bar{x} \oplus y = F \Rightarrow F = \bar{x}y + x\bar{y}$ (If $y=1 \Rightarrow F=\bar{x}$) $\Rightarrow$ Acts as inverter.

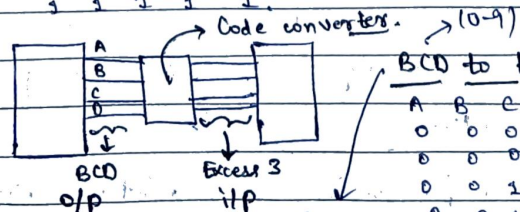
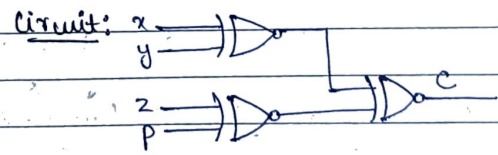
Odd Parity Checker: 4 bits are received (3 i/p + 1 P).

x	y	z	P	C
0	0	0	0	1
0	0	0	1	0
0	0	1	0	0
0	0	1	1	1
0	1	0	0	0
0	1	0	1	1
0	1	1	0	1
0	1	1	1	0
1	0	0	0	0
1	0	0	1	1
1	0	1	0	1
1	0	1	1	0
1	1	0	0	1
1	1	0	1	0
1	1	1	0	0
1	1	1	1	1

if wrong (even no. of 1), then circuit becomes active.

$$C = \sum m(0, 3, 5, 6, 9, 10, 12, 15)$$

$$= x \odot y \odot z \odot P$$



BCD to Excess-3 code converter:

A	B	C	D	w	x	y	z
0	0	0	0	0	0	1	1
0	0	0	1	0	1	0	0
0	0	1	0	0	1	0	1
0	0	1	1	0	1	1	0
0	1	0	0	0	1	1	1
0	1	0	1	1	0	0	0
0	1	1	0	1	0	0	1
0	1	1	1	1	0	1	0
1	0	0	0	1	0	1	1
1	0	0	1	1	1	0	0
1	0	1	0	1	1	0	1
1	0	1	1	1	1	1	0

(Add 3)

(10-15 are unused - I don't care terms)

	C0			
AB	00	01	11	10
00				
01		2	1	2
11	X	X	X	X
10	2	2	X	X

→ Similarly obtain for X, Y, Z.

(o/p depends only on the present i/p).

$W = BD + A + CB$

Designing Combinational Circuits...

13/08

Small Scale Integration: (SSI) → 10 transistors in 1 chip.

Medium Scale Integration: (MSI) → 100 " " "

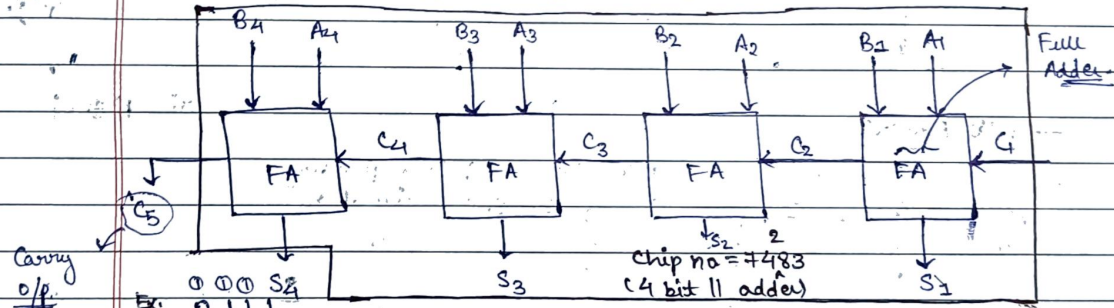
Large Scale Integration: (LSI) 1000 transistors in 1 chip.

Very Large Scale " " (VLSI) 71000 " " " "

Ripple Carry Adder.

$A = A_3 A_2 A_1 A_0$
 $B = B_3 B_2 B_1 B_0$

(parallel & cascaded) → carry from 1 goes to others.



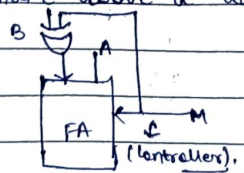
Carry o/p
 Ex: $\begin{array}{r} 0001 \\ 0111 \\ \hline 0001 \\ 1000 \end{array}$

→ let time needed to generate carry for 1 FA be S .

⇒ for n -bit, total time = nS (proportional to n)
 (delay).

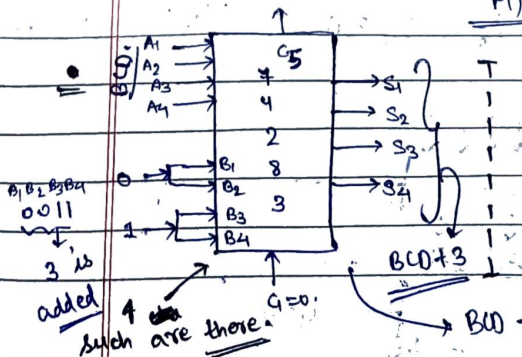
Delay is proportional to the no. of bits you want to add. (Disadvantage).

→ To make above a adder as well as subtractor:



when $M=0 \Rightarrow$ It acts as adder.

when $M=1 \Rightarrow$ It acts as subtractor using 2's complement. (XOR gives 2's complement, 1 from M).

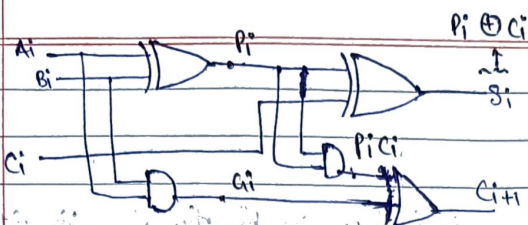


Look-Ahead Carry Adder: 0111.

Time Complexity

Hardware Complexity increases in this.

BCD to excess-3



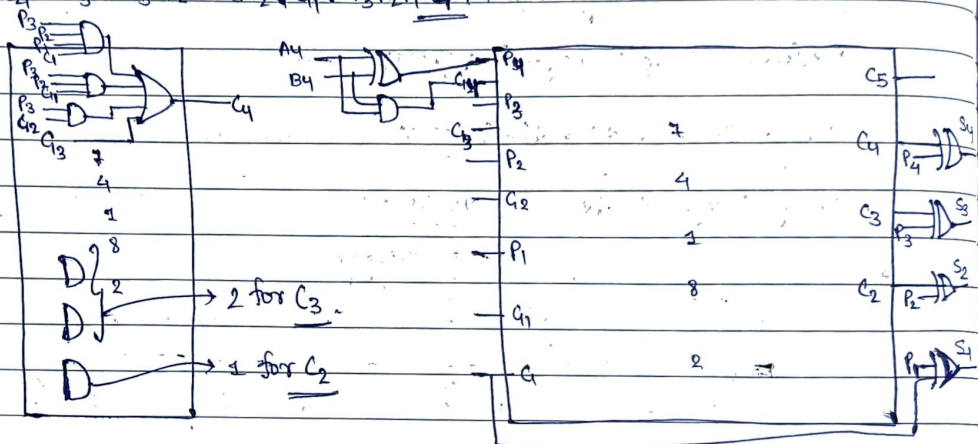
$G_i = A_i B_i \Rightarrow$ Carry Generate.

$$C_{i+1} = G_i + P_i C_i$$

$$P_i = A_i \oplus B_i$$

$$C_2 = G_1 + P_1 C_1, \quad C_3 = G_2 + P_2 C_2 = G_2 + P_2 (G_1 + P_1 C_1) = G_2 + P_2 G_1 + P_2 P_1 C_1$$

$$\Rightarrow C_4 = G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 C_1$$



$$S_i = P_i \oplus C_i$$

→ BCD Adder:

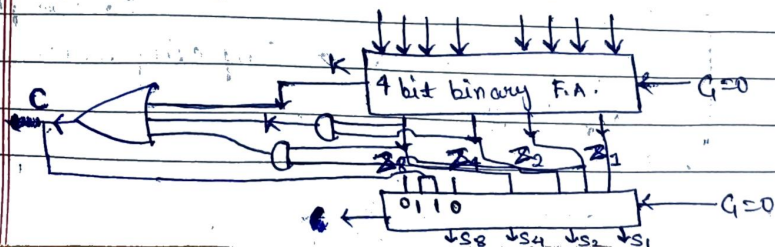
Binary Sum					BCD Sum				Corresponding decimal no.
K	Z ₈	Z ₄	Z ₂	Z ₁	C	S ₄	S ₃	S ₂	
0	0	0	0	0	0	0	0	0	0
1	0	0	0	1	0	0	0	1	1
2	0	0	1	0	0	0	1	0	2
3	0	0	1	1	0	0	1	1	3
4	0	1	0	0	0	1	0	0	4
5	0	1	0	1	0	1	0	1	5
6	0	1	1	0	0	1	1	0	6
7	0	1	1	1	0	1	1	1	7
8	1	0	0	0	1	0	0	0	8
9	1	0	0	1	1	0	0	1	9
10	1	0	1	0	1	0	1	0	10
11	1	0	1	1	1	0	1	1	11
12	1	1	0	0	1	1	0	0	12
13	1	1	0	1	1	1	0	1	13
14	1	1	1	0	1	1	1	0	14
15	1	1	1	1	1	1	1	1	15

• from 0-9, both are same, when binary sum > 9 we have to add 6 to obtain BCD Sum.

$$\begin{aligned} \text{Ex } 15 &= 01111 \\ &+ 0110 \rightarrow 6 \\ \hline &10101 \\ &\text{BCD} \end{aligned}$$

How to identify: (when to add 6)? $C = K + Z_8 Z_4 + Z_8 Z_2$

K must be 1 (16, 17, 18, 19), or Z_8 and Z_4 both are 1 (12, 13, 14, 15) or Z_8 and Z_2 must be 1 (10, 11).



- Magnitude Comparator: $A, B \rightarrow A=B, A>B \text{ or } A<B$. $\downarrow$ o/p.
(Start from MSB $\rightarrow$ move towards LSB).

for $A=B$: XNOR gate is used. (XNOR = $xy + x'y'$). $X_i = A_i B_i + A_i' B_i'$

$A = A_3 A_2 A_1 A_0$, $B = B_3 B_2 B_1 B_0$. (for equal: $A_3=B_3$; $A_2=B_2$; $A_1=B_1$; $A_0=B_0$).

$$(A=B) = X_3 \cdot X_2 \cdot X_1 \cdot X_0$$

for $A>B$: $A_3 \bar{B}_3 + X_3 A_2 \bar{B}_2 + X_3 X_2 A_1 \bar{B}_1 + X_3 X_2 X_1 A_0 \bar{B}_0$. $\rightarrow$ If $X_1, X_2, X_3 = 1$.

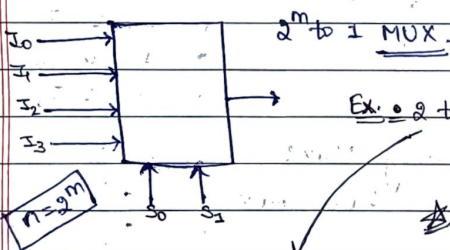
for $A<B$: $\bar{A}_3 B_3 + X_3 \bar{A}_2 B_2 + X_3 X_2 \bar{A}_1 B_1 + X_3 X_2 X_1 \bar{A}_0 B_0$.

- $\rightarrow$ Can be implemented using basic gates (AND, OR, NOT) as well as through (XOR, NAND).

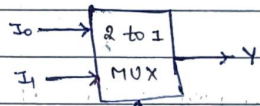
- Generalized Logic Circuit: Building Blocks-

① Multiplexer ② Decoder/Demultiplexer ③ Encoder.

- ① Multiplexer: It is an electronic switch that has n -input lines and m -select lines and 1 output line.



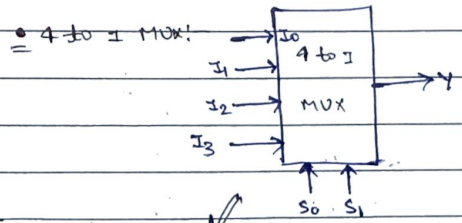
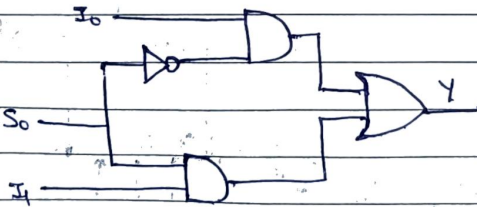
Ex: 2 to 1 line MUX:



$$\begin{cases} \text{If } S_0 = 0 \Rightarrow I_0 \text{ to } Y \\ \text{If } S_0 = 1 \Rightarrow I_1 \text{ to } Y \end{cases} \Rightarrow Y = \bar{S}_0 I_0 + S_0 I_1$$

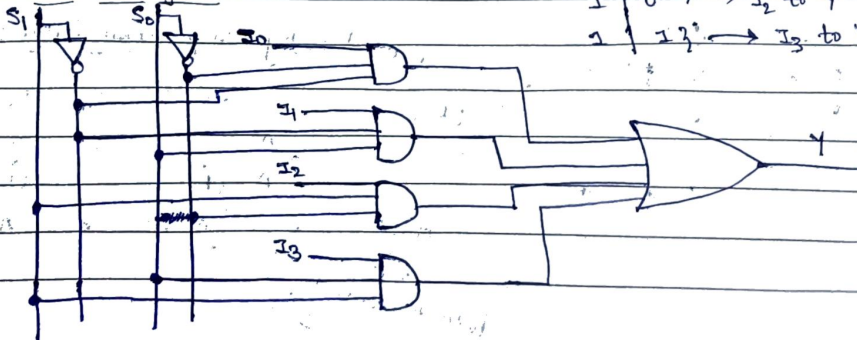
when 1
 $\uparrow$
when 0

Circuit Diagrams:



$$Y = \bar{S}_1 \bar{S}_0 I_0 + \bar{S}_1 S_0 I_1 + S_1 \bar{S}_0 I_2 + S_1 S_0 I_3$$

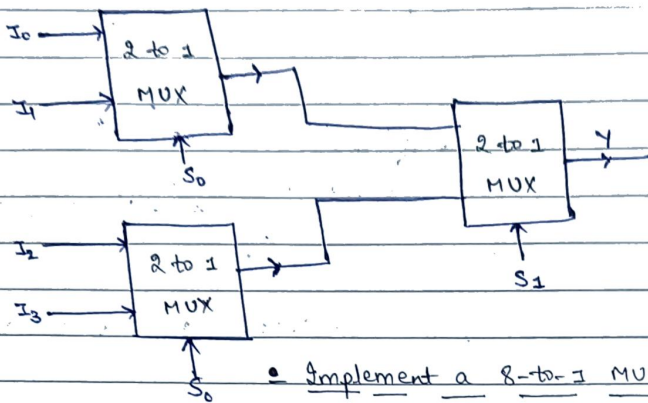
Circuit Diagram:



S_1	S_0	
0	0	$\rightarrow I_0$ to Y
0	1	$\rightarrow I_1$ to Y
1	0	$\rightarrow I_2$ to Y
1	1	$\rightarrow I_3$ to Y

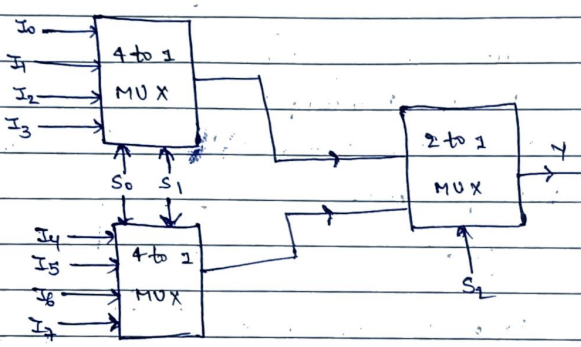
Connections

- Implement a 4-to-1 MUX using (2-to-1 MUX): 3 will be needed



S_1	S_0	Y
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3

- Implement a 8-to-1 MUX using smaller MUX:



S_2	S_1	S_0	Y
0	0	0	I_0
0	0	1	I_1
0	1	0	I_2
0	1	1	I_3
1	0	0	I_4
1	0	1	I_5
1	1	0	I_6
1	1	1	I_7

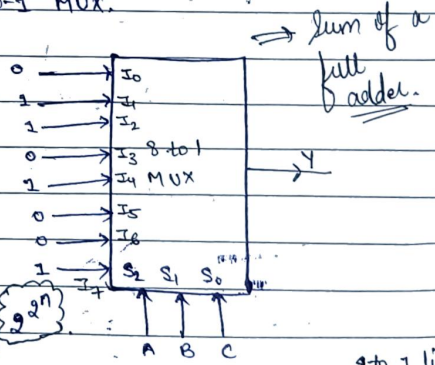
Implementation of logic function using MUX:

- Implement n-variable function using 2^n -to-1 MUX.

Ex: A, B, C, F

$\Rightarrow$ We need $2^3 = 8$ -to-1 MUX.

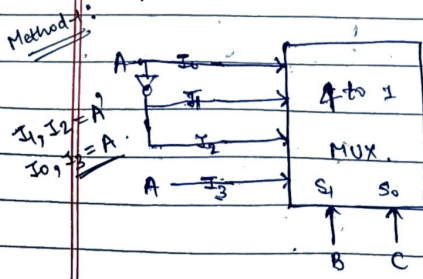
A	B	C	F
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1



Total no. of possible combination of function: 2^{2^n}
(n=no. of variables).

for complete full adder $\Rightarrow 2$ MUX.
(1 sum + 1 carry)

- Implement above circuit using 2^n -to-1 MUX:



Method 2: By Implication Table:

OR:

Circle the minterms, then do column wise.

I_0	I_1	I_2	I_3
0	1	1	0
1	0	0	1
A'	A	A'	A

Ex. If $F(A, B, C) = \Sigma(1, 3, 5, 6)$.

	I_0	I_1	I_2	I_3
A'	0	①	2	③
A	4	⑤	⑥	7
	0	1	A	A'

I_0 connected to 0, I_1 to 1,
 I_2 to A, I_3 to A' .

In a column, if none is selected $\Rightarrow 0$, if both selected $\Rightarrow 1$, if any one then that only.

→ Implementation of arbitrary function using 2-to-1 MUX:

We repeatedly split a function into smaller sub-functions using Shannon's Decomposition Theorem. The theorem states -

- Any n-variable function $f(x_1, x_2, \dots, x_n)$ can be decomposed with respect to any of the n-variables, (say x_1) as $f(x_1, x_2, \dots, x_n) = \bar{x}_1 f(0, x_2, x_3, \dots, x_n) + x_1 f(1, x_2, x_3, \dots, x_n)$.

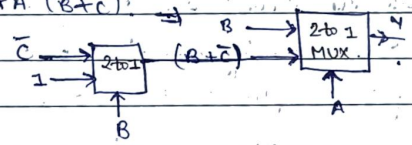


Ex. $f = \bar{A}B + BC + A\bar{C}$.

Applying thm w.r.t. A: $f = \bar{A}(B + BC) + A(\bar{C} + BC)$

$$\Rightarrow f = \bar{A}B + AB + AC = \bar{A}B + A(B + \bar{C})$$

Again applying thm on $(B + \bar{C})$ w.r.t. B:



$$\Rightarrow \bar{B}(\bar{C}) + B(C) \neq$$

→ Quine-McCluskey Method: needed when no. of variables is more. (≥ 5).
Way to reduce boolean expression. It has 2 steps: (1) Obtain all the prime implicants. ($xy + x\bar{y} = x$).

Ex. $\bar{A}\bar{B}\bar{C}\bar{D} + \bar{A}\bar{B}C\bar{D} = \underline{101}$, $f(a, b, c, d) = \Sigma m(0, 1, 2, 5, 6, 7, 9, 10, 14)$

1	0	1	0
A	B	C	D

0	0000	✓	5	0101	✓	7	0111	✓
1	0001	✓	6	0110	✓	14	1110	✓
2	0010	✓	9	1000	✓			
8	1000	✓	10	1010	✓			

(not in same group). Compare among adjacent groups. 4 groups are formed. In every grp, same no. of 1 is there.

Combine: $(0, 1) \Rightarrow 000$, $(1, 5) \Rightarrow 0-01$, $(5, 7) \Rightarrow 01-1\bar{b}d$,
 $(10, 2) \Rightarrow 00-0$, $(1, 9) \Rightarrow -001$, $(6, 7) \Rightarrow 011\bar{a}bc$,
 $(0, 8) \Rightarrow -000$, $(2, 6) \Rightarrow 0-10$, $(6, 14) \Rightarrow -110$,
must vary only at 1. $(2, 10) \Rightarrow -010$, $(10, 14) \Rightarrow 1-10$,
 $(8, 9) \Rightarrow 100-$,
 $(8, 10) \Rightarrow 10-0$.

New 3 groups are there.

Repeat same again: $(0, 1, 8, 9) \Rightarrow -\bar{b}\bar{c}$, $(2, 6, 10, 14) \Rightarrow -\bar{c}\bar{d}$,
 $\bar{b}\bar{d}(0, 2, 8, 10) \Rightarrow -0-0$, $XX(2, 10, 6, 14) \Rightarrow -10$,
 $XX(0, 8, 1, 9) \Rightarrow -00-$,
 $XX(0, 8, 2, 10) \Rightarrow -00-$. Remove duplicates.

$\Rightarrow$ All prime implicants $\equiv$ Unselected minterms + from last groups.

$$\Rightarrow \bar{a}\bar{c}\bar{d} + \bar{a}b\bar{d} + \bar{a}b\bar{c} + \bar{b}\bar{c} + \bar{b}\bar{d} + \bar{c}\bar{d}$$

② Prime Implicant Chart/ Table: All prime implicants become ~~columns~~ ^{rows} of table. minterms become column. $\Rightarrow \Sigma m(0,1,2,3,6,7,8,9,10,14)$.

Table:

minterms $\rightarrow$	0	1	2	3	6	7	8	9	10	14
(0,1,8,9) $\bar{b}\bar{c}$	X	X					X	(X)		
(0,2,8,10) $\bar{b}\bar{d}$	X		X				X		X	
(2,6,10,14) $c\bar{d}$			X		X				X	(X)
(1,5) $\bar{a}\bar{c}d$		X		X						
(3,7) $\bar{a}bd$				X		X				
(6,7) $\bar{a}bc$						X	X			

prime implicants $\rightarrow$

$\rightarrow$ If a minterm is covered by only 1 prime implicant then that prime implicant is called the essential prime implicant and must be included in the minimized expression.

So, from above: essential prime implicants $= \bar{b}\bar{c}, c\bar{d}$.

Then, check if essential covers all the minterms. Then select 1 or more prime implicant that covers all minterms (remaining). $\Rightarrow \bar{a}bd$.

$$\Rightarrow f = \bar{b}\bar{c} + c\bar{d} + \bar{a}bd$$

By K-map:

same answer $\Rightarrow$ Verified.

Verifying:

ab \ cd	00	01	11	10
00	1	1		1
01		1	1	1
10				1
11	1	1		1

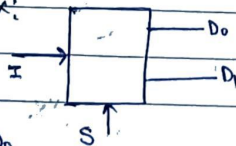
$\bar{b}\bar{c}$ (covers 00, 11)
 $c\bar{d}$ (covers 01, 11)
 $\bar{a}bd$ (covers 01, 11)

② Demultiplexer: (Decoder). It is an electronic switch that works in a reverse manner as compared to multiplexer. It has 1 input line, n output line and m-select lines.

$$n = 2^m$$

1-to- 2^m Demultiplexer:

1-to-2 DEMUX:



Truth Table:

I	S	D ₀	D ₁
0	0	0	0
0	1	0	0
1	0	1	0
1	1	0	1

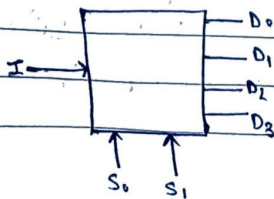
Expression

$$\Rightarrow D_0 = I\bar{S}, D_1 = IS$$

If S=0, I to D₀

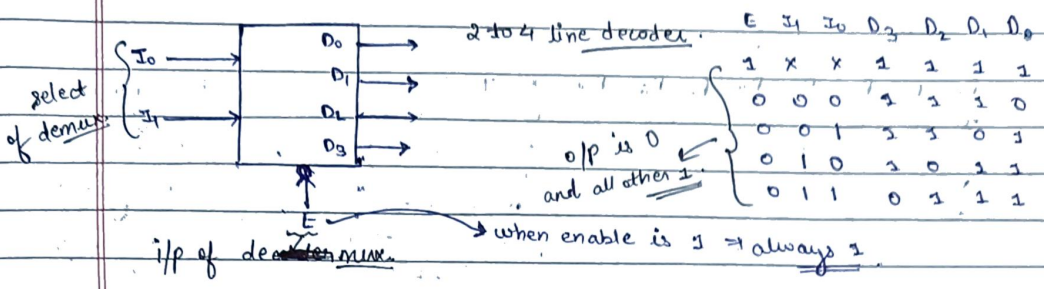
"S=1, " " D₁

1-to-4 DEMUX:

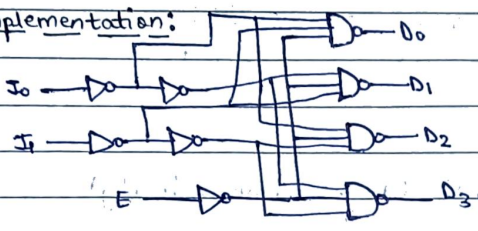


I	S ₁	S ₀	D ₃	D ₂	D ₁	D ₀	Expression
0	0	0	0	0	0	0	$D_0 = I\bar{S}_1\bar{S}_0$
0	0	1	0	0	0	0	$D_1 = I\bar{S}_1S_0$
0	1	0	0	0	0	0	$D_2 = IS_1\bar{S}_0$
0	1	1	0	0	0	0	$D_3 = IS_1S_0$
1	0	0	0	0	0	1	
1	0	1	0	0	1	0	
1	1	0	0	1	0	0	
1	1	1	0	1	0	0	

→ Decoder: It is a special case of DEMUX. It has n input lines and 2^n output lines. Depending upon application of i/p, exactly 1 o/p is set to 1, and all other o/p are set to 0.

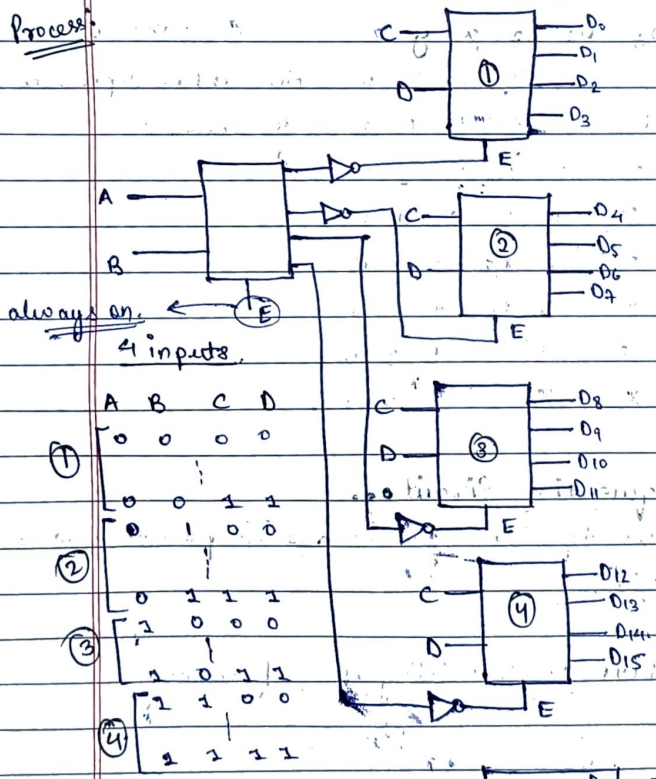


Implementation:



Q: Is it possible to make a 4-to-16 DECODER using 2-to-4 DECODERS? Yes.
 Total 5 Decoders will be needed.

Process:

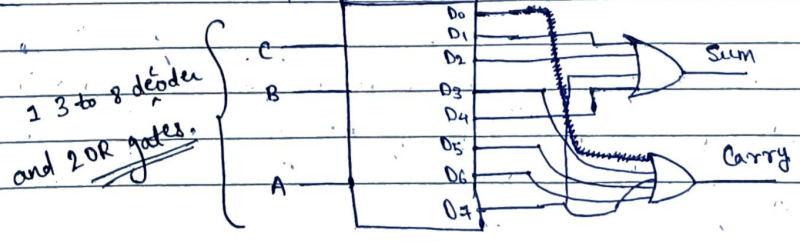


Q: Is it possible to implement boolean expression (full adder circuit) using decoder? Yes.
 full adder $\Rightarrow$ 3 i/p, 8 o/p.

A	B	C	S	C
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

$S = \sum m(1, 2, 4, 7)$
 $C = \sum m(3, 5, 6, 7)$

3 to 8 decoder.



⑧ Encoder: An encoder typically has 2^n input lines and n output lines. Only 1 i/p line is at logic level 1 at a particular time. The o/p lines contain the binary code corresponding to the i/p line which is at 1.

• 8-to-3-line encoder:

D_0	D_1	D_2	D_3	D_4	D_5	D_6	D_7	x	y	z	
1	0	0	0	0	0	0	0	0	0	0	$\rightarrow 0$
0	1	0	0	0	0	0	0	0	0	1	$\rightarrow 1$
0	0	1	0	0	0	0	0	0	1	0	$\rightarrow 2$
0	0	0	1	0	0	0	0	0	1	1	$\rightarrow 3$
0	0	0	0	1	0	0	0	1	0	0	$\rightarrow 4$
0	0	0	0	0	1	0	0	1	0	1	$\rightarrow 5$
0	0	0	0	0	0	1	0	1	1	0	$\rightarrow 6$
0	0	0	0	0	0	0	1	1	1	1	$\rightarrow 7$

Encoder

Has certain

limitations

of no use.

Priority Encoder:

 D_i, D_j If $i > j \Rightarrow D_i > D_j$ (D_i has more priority).

• 8-to-3-line encoder:

D_0	D_1	D_2	D_3	D_4	D_5	D_6	D_7	x	y	z	
1	0	0	0	0	0	0	0	0	0	0	
x	1	0	0	0	0	0	0	0	0	1	
x	x	1	0	0	0	0	0	0	1	0	
x	x	x	1	0	0	0	0	0	1	1	
x	x	x	x	1	0	0	0	1	0	0	
x	x	x	x	x	1	0	0	1	0	1	
x	x	x	x	x	x	1	0	1	1	0	
x	x	x	x	x	x	x	1	1	1	1	

 $x \rightarrow$ either 0 or 1.

Now, obtain expression

for x, y, z .

03/09

Sequential Circuit...

In these circuits, o/p not only depends on input but also on present state (or past history) of the circuit.

• LATCHES and FLIP-FLOPS: Basic building block of memory elements.

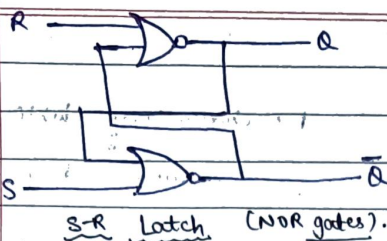
Synchronous

Asynchronous

- A latch is a temporary storage device that has 2 stable states, 0 and 1.
- A flip-flop is a special type of latch where a ^{clock} ~~flip~~ signal figures the changes in stored value.

triggers

③ while checking for o/p consider previous state of circuit $\rightarrow$ for checking for (0,0) also see o/p of previous condition.



Types:

- (i) S-R
- (ii) D (Delay)
- (iii) J-K
- (iv) T (toggle)

Latches and flip flops.

① S-R latch: It is a pair of cross-coupled NAND or NOR gate. It has 2 i/p (R and S) and 2 o/p (Q and Q-bar), o/p are complement to each other.

(If any i/p = 1 for NOR $\Rightarrow$ o/p = 0)

S	R	Q	Q-bar
1	0	1	0
0	0	1	0
0	1	0	1
0	0	0	1

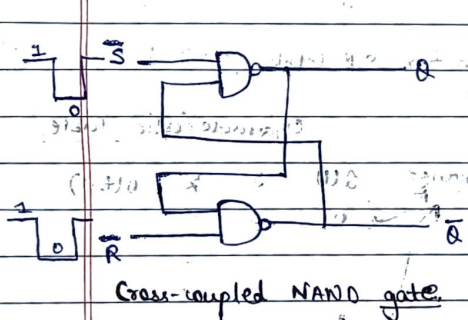
no change.

xx { 1 1 0 0 } $\rightarrow$ violates

We don't use (1,1) combination in S-R latch.

after this if $S=R=0 \Rightarrow$ o/p = 1, 1 if and only if speed of both gates is same which is practically not possible.

not predictable. (as o/p oscillates).



S	R	Q	Q-bar
1	0	0	1
1	0	1	0
0	1	1	0
1	1	1	0
0	0	1	1

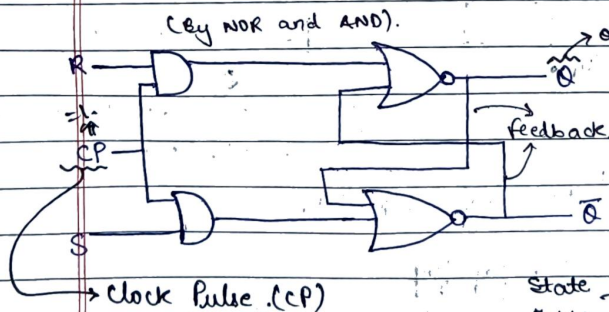
no change

this case also it depends on speed of gate.

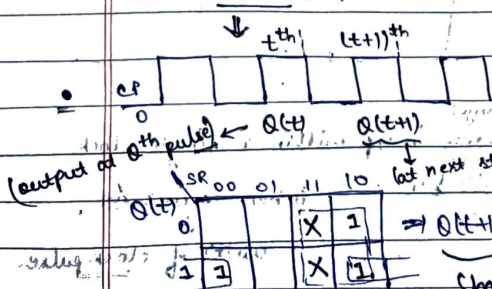
invalid.

(you will then get NOR table).

$\rightarrow$ S-R flip-flop: S-R latch can be modified by providing additional i/p that determines when the state of the circuit is to be changed.



$\rightarrow$ When $CP=0 \Rightarrow$ o/p of 2 AND gates is 0 $\rightarrow$ no change when R, S have a certain value (in NOR).



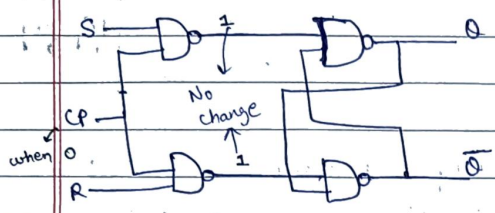
State Table	Q(t)	S	R	Q(t+1)
0	0	0	0	0
0	0	0	1	0
0	1	0	1	1
0	1	1	1	(indeterminate)
1	0	0	1	1
1	0	1	0	0
1	1	0	1	1
1	1	1	1	(i.d.)

not truth table.

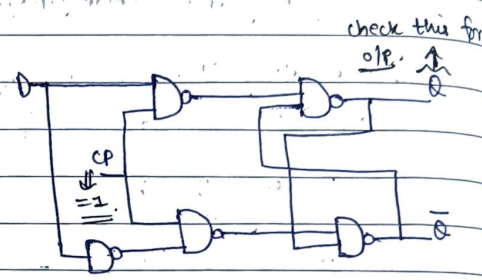
Characteristic eqn of flip flop.

$$Q(t+1) = S + \bar{R}Q(t)$$

Using NAND: Replace all by NAND. Then when $CP = 0 \Rightarrow$ o/p = 1, 1 for both
NAND = no change.



for characteristic table
 Always $CP = 1$.



② D-flip-flop: Modification of S-R flip flop

$Q(t)$	D	$Q(t+1)$
0	0	0
0	1	1
1	0	0
1	1	1

Equation:
 $Q(t+1) = D$

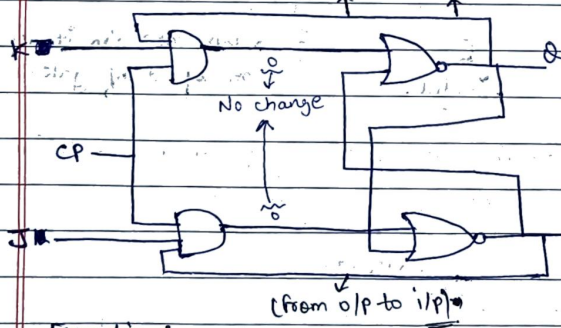
Circuit Diagram.

Input is only the i/p but we put delay that's why name is D-flip-flop.

Imp. ④ Why $\begin{smallmatrix} S \\ 1 \end{smallmatrix} \begin{smallmatrix} R \\ 1 \end{smallmatrix}$ is not a valid i/p combination for S-R latch. $\rightarrow$ Imp. for exam.

③ J-K flip-flop: Modification of S-R
 (feedback from o/p)

Characteristic Table.



$Q(t)$	J	K	$Q(t+1)$
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	0

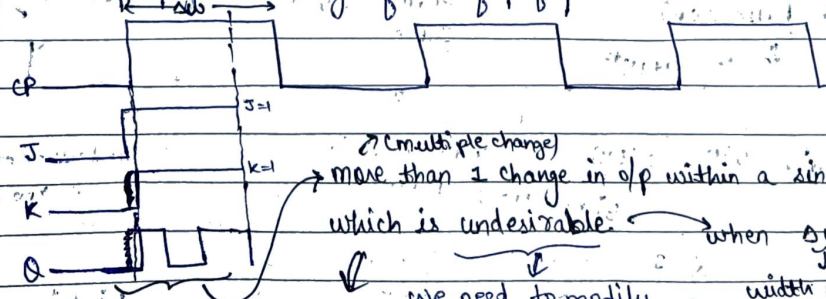
Equation:

$Q(t)$	00	01	11	10
0			1	1
1	1	1		1

$Q(t+1) = J\bar{Q}(t) + KQ(t)$

whenever both $J, K = 1$ then o/p is inverted. ($Q(t)$ & complement).

$\rightarrow$ let $\Delta t =$ propagation delay of J-K flip-flop.



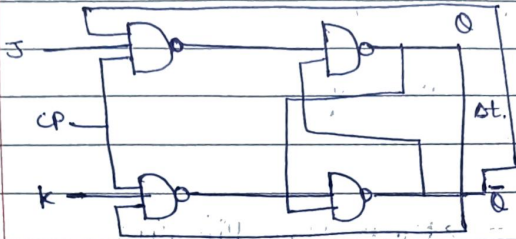
Multiple change
 more than 1 change in o/p within a single clock
 which is undesirable.

when $\Delta w > \Delta t$
 width of clock pulse.

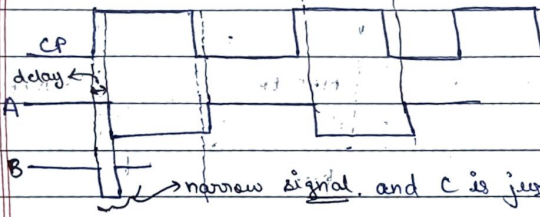
Problem we need to modify.
 of J-K flip-flop.

- 3 ways to modify:-
- ① Ensure $\Delta w < \Delta t$ } not in our hand. (using narrow clock)
 - ② Edge Triggered - flip-flop. (Circuit is triggered either in +ve or -ve direction)
 - ③ Master-slave flip-flop. (when 1 is active, other is inactive). (2 flip-flops)

NAND Version:

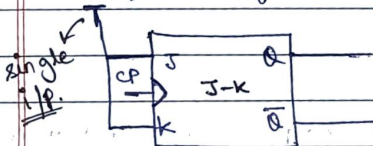


Generating narrow clock signal: → CP → A → B → C



You can use odd no. of inverters for more delay and same o/p.

④ T-flip-flop: Modification of J-K.



Q(t)	T	Q(t+1)
0	0	0
0	1	1
1	0	1
1	1	0

When $T=0 \Rightarrow$ no change.

When $T=1 \Rightarrow$ toggling occurs.

and $Q(t+1)$ is complement of $Q(t)$.

Toggle-flip-flop.

$$Q(t+1) = TQ(t) + T\bar{Q}(t)$$

→ The state of a flip-flop is changed by momentary change in the i/p signal. This momentary change is called a trigger.

• ~~clock~~ flip flops are triggered by pulse.

↑ ↓ ⇒ falling edge ⇒ -ve edge triggered.

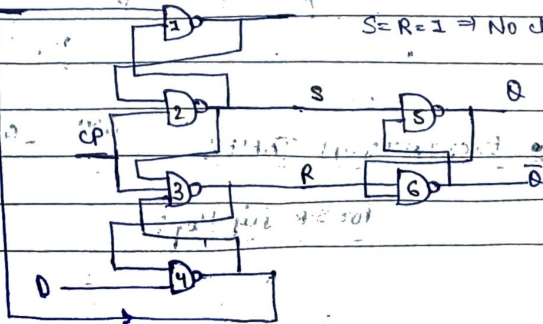
if triggered at rising edge of clock ⇒ +ve edge triggered.

Edge Trigger D-flip-flop:

(B. Latches) Logic Diagram: →

+ve edge triggered

S	R	Q	Q-bar
1	1	no change	
0	1	1	0
1	0	0	1



$S=R=1 \Rightarrow$ No change.